Water Quality Prediction: Energy-Aware Offline Multimodal Neuromorphic Edge Inference

Felipe de Vasconcelos Assunção[1], Alfredo Paulo Oliveira Barros[1], Reyner Carlos Silva Alegria[1], Elton dos Santos Araújo[1], Alex Martins Ramos[1], Vicente Ferreira de Lucena Junior[1], Vandermi João da Silva[1]

[1]Universidade Federal do Amazonas (UFAM)

felipe.assuncao@ufam.edu.br, alfredo.barros@ufam.edu.br, reyner.alegria@ufam.edu.br, vicente@ufam.edu.br, elton_araujo_@hotmail.com, alexmartins@ufam.edu.br, vandermi@ufam.edu.br

Abstract

This work investigates energy-aware neuromorphic edge inference for multimodal water quality prediction in a remote Amazonian region of northern Brazil. A spiking neural network (SNN) is deployed on low-cost embedded platforms to jointly predict pH, dissolved oxygen, and turbidity directly from georeferenced physicochemical measurements, without reliance on cloud connectivity. To mitigate the limited size of the field dataset, light Gaussian augmentation is employed to improve training stability. The SNN architecture, based on a leaky integrate-and-fire (LIF) hidden layer, is optimized through systematic grid search over temporal steps, learning rate, threshold, decay factor, and hidden-layer size. The final model achieves competitive predictive performance while enabling fully offline inference on constrained hardware. Latency and energy benchmarks are reported across Raspberry Pi, Android devices, and ESP32 platforms, including INA219-based power telemetry and incremental energy-per-inference estimates derived from idle and load measurements. Results demonstrate the feasibility of neuromorphic inference for autonomous environmental monitoring, revealing measurable energy-latency trade-offs and supporting deployment in connectivity- and power-limited scenarios.

Index Terms— Spiking Neural Networks, edge AI, water quality, offline inference, neuromorphic computing, environmental monitoring.

1 INTRODUCTION

The Amazon basin holds the largest surface freshwater reserve on the planet and plays a key role in climate regulation and global biodiversity. Monitoring water quality in this region is increasingly challenging due to vast distances, dense forest cover, complex hydrography, and limited communication infrastructure. Traditional manual sampling and laboratory analysis, while accurate, are expensive, logistically difficult, and unable to provide the temporal resolution needed to detect sudden contamination events (e.g., oil spills or illegal discharges) or track hydrological changes driven by extreme climate events.

This context of environmental monitoring under energy and connectivity constraints is also highlighted in the position paper by Aral (2024), which outlines directions for neuromorphic Edge AI in rural scenarios. In response, the convergence of the Internet of Things (IoT) and Artificial Intelligence (AI) has been proposed as a viable path for large-scale remote monitoring (RAHU et al., 2023). Connected sensors can stream data in real time, enabling more proactive water resource management. However, continuous transmission of raw data to the cloud faces severe connectivity limitations in the Amazon, resulting in high energy consumption and unacceptable latency. A practical alternative is to move intelligence to the edge, where data are processed locally on the monitoring device.

The critical challenge, as noted by Rahu et al. (2023), is that high-performance machine learning models such as Long Short-Term Memory (LSTM) networks demand computational and energy resources that often exceed the capacity of low-cost microcontrollers used in battery- or solar-powered sensor nodes. Although LSTMs perform well in complex hydrological time series modeling (PYO et al., 2023), their computational cost can reduce the autonomy of remote stations. In this setting, neuromorphic computing—and specifically SNNs—offers a promising alternative. SNNs process information asynchronously through sparse spikes, activating only when significant input variations occur. This property can provide orders-of-magnitude energy savings compared to conventional artificial neural networks (ANNs) that operate continuously.

This paper evaluates the technical and practical feasibility of SNN-based edge inference for predicting water-quality parameters. We focus on multimodal prediction of pH, dissolved oxygen, and turbidity, and on validating the full pipeline in real conditions, from data processing to offline deployment. The remainder of this paper is organized as follows: Section 2 presents related work and positions the study; Section 3 details materials and methods, including data, model, training, and mobile deployment; Section 4 presents results and performance analyses; Section 5 discusses limitations and implications; and Section 6 concludes the paper and outlines future directions.

2 RELATED WORK

2.1 Deep learning, hybrid models, and reliability in water quality

Water quality modeling has evolved from classical approaches to deep neural networks, with recurrent models standing out in environmental time series. Pyo et al. (2023) report consistent gains with LSTM in inland waters, while Qi, Huang e Wang (2020) and Liu et al. (2019) reinforce the effectiveness of recurrent architectures and IoT for predicting physicochemical parameters. Recent reviews confirm the dominance of deep learning for nonlinear hydrological patterns (YAN et al., 2024; PAN et al., 2025). In parallel, hybrid approaches seek robustness under noise; Wu e Wang (2022) integrate ANN, Wavelet, and LSTM, and comparative studies evaluate multiple algorithms for water-quality parameters (AHMED et al., 2019; KALAIVANAN; VELLINGIRI, 2022; AL-ADHAILEH; ALSAADE, 2021). These results show accuracy gains but increase preprocessing and inference cost when local execution on constrained hardware is required. Instrument reliability is also critical: El-Shafeiy et al. (2023) address real-time anomaly detection, while Lamrini et al. (2024) evaluate outliers in smart sensing; IoT system reviews highlight that performance depends on data architecture, calibration, and sensor-network robustness (ZULKIFLI et al., 2022; MILLER et al., 2025).

2.2 Edge AI, neuromorphic computing, and SNN fundamentals

Edge AI has been proposed to reduce latency and cloud dependence in water monitoring (RAHU et al., 2023; LOWE; QIN; MAO, 2022). Applied studies in sensor networks and remote monitoring show operational feasibility but also highlight hardware, energy, and connectivity limits (ROSERO-MONTALVO et al., 2020; ALPROL et al., 2024). In the Amazon, these constraints motivate local inference architectures with lower computational cost. Strategic literature emphasizes that neuromorphic edge architectures can reduce energy consumption and improve resilience in low-infrastructure areas (ARAL, 2024); recent results in embedded Industrial Internet of Things (IIoT) also report energy gains with neuromorphic hardware (AZADI et al., 2025), while ESP32-based solutions demonstrate real-time monitoring feasibility (AL-SHAREEDA et al., 2025). To mitigate the energy cost of conventional ANN inference, neuromorphic computing relies on event-driven processing. In SNNs, information is encoded as asynchronous spikes, reducing computation when inputs are stable. SNN accelerators with compute-in-memory report both performance and energy efficiency gains (HUIJBREGTS et al., 2024), and recent reviews consolidate modeling, training, and latency/energy metrics (JR., 2025). Among bio-inspired models, Chadalavada et al. (2025) propose Gated Liquid Neural Networks for water-quality indices, but still in continuous regimes; SNNs exploit event-driven inference and are potentially better aligned with edge constraints.

Among models that describe spiking neuron dynamics, Leaky Integrate-and-Fire (LIF) offers a balance between biological plausibility and computational efficiency. The membrane potential $V(t)$ follows the linear differential equation:

$$\tau_m \frac{dV(t)}{dt} = -(V(t) - V_{rest}) + R_m I(t)$$

Here, τ_m is the membrane time constant, V_{rest} the resting potential, R_m the membrane resistance, and $I(t)$ the synaptic input current. A spike is emitted when $V(t)$ crosses a threshold V_{th}, and the potential is reset to V_{reset}.

For digital implementations (e.g., microcontrollers), the continuous model is discretized as:

$$V[t+1] = V[t] + \frac{1}{\tau_m}(-(V[t] - V_{rest}) + I[t])\Delta t$$

This formulation highlights a key advantage for edge deployment: when inputs are stable ($I[t] \approx 0$), the system evolves with low activity, favoring energy efficiency on constrained devices.

2.3 Local context and positioning of this work

Within the broader Amazonian environmental context, Lago do Serpa represents a relevant case study where local territorial and ecological dynamics reinforce the need for continuous monitoring (NOGUEIRA; RIBEIRO; RIBEIRO, 2022). The experiment was conducted in the Amazon region (Itacoatiara, AM, Brazil), in a lentic environment characterized by complex hydrological dynamics and marked seasonal variability. Data collection relied on in-situ measurements and georeferenced sampling points distributed across the lake to capture spatial gradients and local heterogeneity in physicochemical parameters (Figure 1). The sampling design emphasizes spatial coverage rather than dense temporal repetition at a single location, aligning with the objective of producing spatially interpretable maps and identifying potential micro-zones of degradation. This spatial

emphasis is particularly relevant in Amazonian settings, where access is logistically constrained and localized processes (e.g., sediment resuspension, small inflows, or human activity near the margins) can significantly alter water quality over short distances.

Classic ANN-based water-quality forecasting demonstrated modeling potential in different scenarios (PALANI; LIONG; TKALICH, 2008), but without a focus on neuromorphic edge inference. LSTM and remote sensing studies report accuracy gains and thematic maps in large lakes (QI; HUANG; WANG, 2020), while IoT frameworks and wireless sensor networks prioritize acquisition and prediction but typically depend on continuous connectivity (RAHU et al., 2023; ROSERO-MONTALVO et al., 2020). Bio-inspired models such as Liquid Neural Networks achieve promising results for water-quality indices but operate in continuous regimes and do not fully exploit event-driven inference (CHADALAVADA et al., 2025). Strategic Edge AI studies highlight the potential of SNNs for rural environments, yet often lack complete experimental validation under real hardware and energy constraints (ARAL, 2024).

This work positions itself by integrating (i) multimodal prediction of pH, dissolved oxygen, and turbidity; (ii) spatial analysis and pollution mapping; and (iii) fully offline deployment on edge devices with latency and energy benchmarking. This integrated approach remains underexplored in the literature and directly addresses practical deployment gaps in remote environmental monitoring scenarios characterized by limited connectivity and constrained energy resources.

3 MATERIALS AND METHODS

3.1 Data, preprocessing, and split

Data were collected from point measurements in Lago do Serpa (Itacoatiara, AM) and organized in CSV with coordinates (x, y) and physicochemical variables (pH, dissolved oxygen, temperature, apparent color, turbidity, and true color). Cleaning included decimal separator conversion, numerical consistency checks, and removal of incomplete rows. Inputs were defined as $x, y, TEMP, COR_A$, and COR_V; in the univariate model the output was pH, while in the multimodal model the outputs were pH, dissolved oxygen, and turbidity, without including target variables as inputs to avoid leakage. The dataset was georeferenced in Lago do Serpa (Figure 1), and to mitigate overfitting in a small dataset we applied **synthetic data augmentation** (factor 10) with light Gaussian noise to stabilize training.

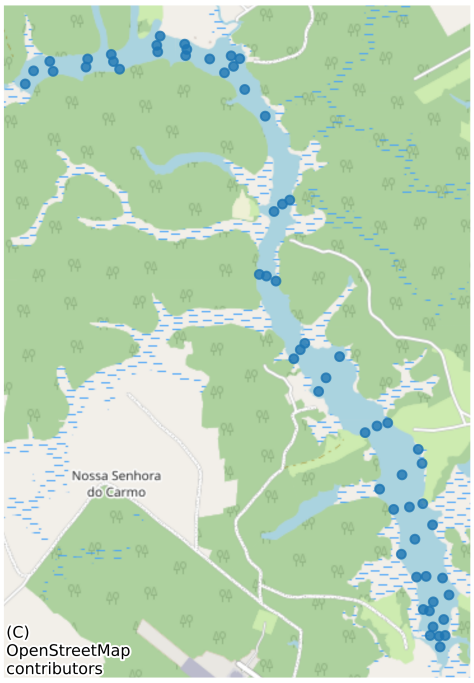


Figure 1: *Sampling locations in Lago do Serpa (Itacoatiara, AM) on an OpenStreetMap basemap. Coordinates were converted to latitude/longitude (WGS84, UTM 21S).*

Inputs were normalized via z-score using training statistics; multimodal outputs were optionally normalized. The train/validation/test split used fixed proportions with seed-controlled shuffling. This split (Table 1) is used in all experiments.

Table 1: *Dataset split.*

Split	Proportion	Number of samples
Train	70%	42
Validation	10%	6
Test	20%	12

3.2 Modeling, training, and evaluation

Normalized inputs were encoded into spike trains using *rate coding*, where firing probability is a sigmoidal function of the input. The SNN used a dense input layer, a hidden layer with Leaky Integrate-and-Fire (LIF) neurons, and a linear output layer. Inference runs over T time steps, accumulating hidden activity, which enables sparse processing; larger T increases temporal resolution but also latency and power consumption. Training used Adam and MSE with *early stopping* and grid search over T, learning rate, threshold, decay, and hidden size, as well as augmentation noise intensity; the best configuration was selected by the lowest validation error. For the multimodal model, we used weighted loss and optional target normalization to

predict pH, dissolved oxygen, and turbidity in a single network. From predictions, we defined a composite pollution index using (i) deviation from neutral pH, (ii) inverse dissolved oxygen, and (iii) normalized turbidity; CONAMA Resolution 357/2005 is used as a **normative reference** to interpret estimated levels, without replacing official indices (RESOLUçãO..., 2005). The index was categorized into three relative levels: low ($[0,0{,}33)$), medium ($[0{,}33,0{,}66)$), and high ($[0{,}66,1]$). Exploratory analyses included boxplots and correlations, while spatial interpretation used point maps and inverse-distance weighting (IDW) surfaces for pH, dissolved oxygen, turbidity, and the pollution index. Model comparison used a multiseed benchmark and a paired sign test on macro metrics, suitable for small samples without normality assumptions.

To make the signal flow explicit (input $\rightarrow$ processing $\rightarrow$ output), the spike encoding and LIF dynamics are defined as follows. Given a normalized input vector $\mathbf{x}$, each feature is converted to a spike probability with a sigmoid:

$$p_i = \sigma(x_i) = \frac{1}{1+e^{-x_i}}. \tag{1}$$

At each time step t, the hidden membrane potential evolves according to the discrete LIF equation:

$$V_h[t+1] = \beta V_h[t] + \sum_i W_{hi} s_i[t] + b_h, \tag{2}$$

where β is the decay, W_{hi} and b_h are weights and bias, and $s_i[t]$ is the encoded spike input. A spike is emitted when $V_h[t+1]$ exceeds a threshold θ; a subtractive reset is applied:

$$s_h[t] = \mathbb{1}(V_h[t+1] > \theta), \quad V_h[t+1] \leftarrow V_h[t+1] - s_h[t]\theta. \tag{3}$$

The output is the temporal average of the linear readout:

$$\hat{\mathbf{y}} = \frac{1}{T}\sum_{t=1}^{T}(W_o \mathbf{s}_h[t] + \mathbf{b}_o). \tag{4}$$

Algorithm 1 summarizes the modeling, training, and evaluation pipeline.

Algorithm 1 Modeling, training, and evaluation pipeline.

Input: raw samples $(x, y, \text{TEMP}, \text{COR_A}, \text{COR_V})$
Normalize inputs with z-score (training statistics).
Encode inputs into spike probabilities via sigmoid (rate coding).
for $t = 1$ to T **do**
    Update LIF membrane potential with decay and synaptic input.
    Generate spikes using threshold θ and apply reset.
    Accumulate readout $W_o \mathbf{s}_h[t] + \mathbf{b}_o$.
end for
Average readout over time to obtain predictions $\hat{\mathbf{y}}$.
Train with MSE (weighted for multimodal outputs) using Adam and early stopping.
Select hyperparameters by grid search (min validation loss).
Evaluate on test split with MAE/RMSE/R^2; run multiseed benchmark and sign test.

3.3 Export and mobile deployment

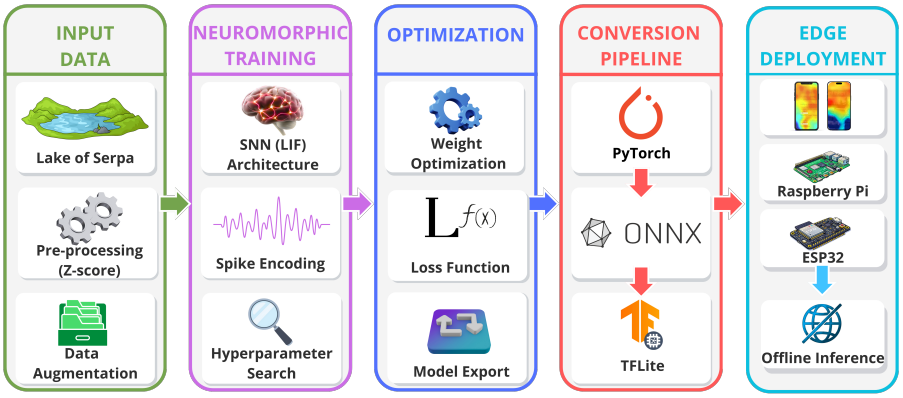
After final training, the multimodal model was exported for mobile inference by serializing PyTorch weights, converting to ONNX while preserving the input/output signature, and transforming to TensorFlow Lite (TFLite) for a lightweight Android runtime. In the Flutter app, the *.tflite* model and normalization parameters (mean and standard deviation) are loaded at startup so each sample is normalized locally before being passed to the TFLite interpreter. Model outputs (pH, dissolved oxygen, turbidity) are de-normalized and used to compute the composite pollution index in real time. For field operations without connectivity, the map is shown offline using MBTiles (Mapbox Tiles), preserving spatial context without internet access.

3.4 Architecture and integration

To assess neuromorphic feasibility on constrained hardware, this study focused on implementing and analyzing SNNs on edge devices. The goal was to evaluate efficiency and accuracy under memory, processing, and energy constraints typical of edge deployment. The proposed SNN uses Leaky Integrate-and-Fire (LIF) neurons to optimize sparse event processing. The architecture (Figure 2) organizes the flow into Input, Processing, and Output stages, exploiting temporal sparsity and activating computation mainly when relevant changes occur, which favors energy efficiency in autonomous systems.

At the Input stage, Lago do Serpa data are normalized (z-score) and augmented to stabilize training. The neuromorphic stage performs temporal encoding (*rate coding*) to convert scalar values into spike trains and runs LIF-based inference. Optimization includes weight and hyperparameter tuning (grid search and *early stopping*), guided by the loss function. After training, the model is exported via PyTorch $\rightarrow$ ONNX $\rightarrow$ TFLite for edge deployment. In operation, the same model runs offline on smartphones, Raspberry Pi, and ESP32, producing multimodal predictions (pH, dissolved oxygen, turbidity), a pollution index, and spatial maps to support field decision-making.

Figure 2: *Proposed neuromorphic architecture for water-quality prediction, highlighting input and preprocessing, neuromorphic training with SNN (LIF) and rate coding, the PyTorch → ONNX → TFLite export pipeline, and offline deployment on edge devices.*



3.5 Hardware specifications for training and inference

To ensure experimental reproducibility and effective validation, a dedicated workstation was used for development, offline training, and preliminary inference simulations. Hardware and software specifications are listed in Table 2.

Table 2: *Workstation specifications used in the experiments.*

Component	Specification
CPU	Intel(R) Core(TM) i7-14650HX (24 threads)
Memory	15 GiB
GPU	NVIDIA GeForce RTX 4060 (8 GB VRAM), Driver 550.163.01, CUDA 12.4
Operating System	Ubuntu 22.04.5 LTS (64-bit)

This environment enabled efficient training with surrogate gradients, which are essential to handle spike non-differentiability prior to portability and optimized deployment on edge devices.

To characterize low-power inference feasibility, the TFLite model was executed on a Raspberry Pi 4B (ARM64) with the XNNPACK delegate (TFLite kernel optimizer), following the export pipeline described in Section 3.7. Device specifications are listed in Table 3.

Table 3: *Inference device specifications (Raspberry Pi 4B).*

Component	Specification
Board	Raspberry Pi 4B
Architecture	ARM64
Operating System	Linux 6.12.34+rpt-rpi-v8-aarch64
Runtime	TFLite (delegate XNNPACK)

Additionally, to evaluate execution on mobile edge devices, the Android app was tested on three phones: *Device A* (model *23021RAAEG*, Qualcomm SoC, Android 15, MemTotal: 5.77 GB), *Device B* (model *SM-A566E*, s5e8855 SoC, Android 16, MemTotal: 7.60 GB), and *Device C* (model *2201117SG*, mt6781 SoC, Android 13, MemTotal: 5.86 GB). These configurations were used for latency and energy benchmarks in the app and are summarized in Table 4.

Table 4: *Android devices used in the experiments.*

Device	Model ID	SoC	Android	RAM
A	23021RAAEG	Qualcomm	15	5.77 GB
B	SM-A566E	s5e8855	16	7.60 GB
C	2201117SG	mt6781	13	5.86 GB

3.6 Energy measurement

To quantify incremental energy during inference, an INA219 current sensor was placed in series with the device power supply and sampled over I^2C. Telemetry was recorded periodically and smoothed with an exponential moving average (EMA), while energy was accumulated over time from the EMA power estimate. Two windows were considered: *idle* (device idle) and *load* (inference benchmark). Average power was computed as $P = V \cdot I$, incremental power as $\Delta P = P_{load} - P_{idle}$, and energy per inference as ΔP/throughput, yielding mJ/inf for cross-platform comparison.

Algorithm 2 Energy measurement procedure using the INA219 sensor.

Initialization:
Configure INA219 and I^2C (calibration 32V/2A).
Initialize serial communication at 9600 baud.
Set telemetry period $\Delta t = 500$ ms and EMA coefficient $\alpha = 0.2$.
Real-time sampling:
while running **do**
  Read bus voltage $V(t)$ and current $I(t)$ from INA219.
  Compute instantaneous power $P(t) = V(t) \cdot I(t)$.
  Update EMA power: $P_{EMA}(t) = \alpha P(t) + (1 - \alpha) P_{EMA}(t - 1)$.
  Accumulate energy: $E = E + P_{EMA}(t) \cdot \Delta t$.
  Send $(V(t), I(t), P_{EMA}(t), E)$ over serial.
end while
Energy computation (host):
Compute mean idle power $\overline{P}_{idle}$ and mean load power $\overline{P}_{load}$.
Compute $\Delta P = \overline{P}_{load} - \overline{P}_{idle}$ and $E_{inf} = \Delta P$/throughput.

For mobile devices, energy was estimated directly in the Android app. Current was obtained via *BatteryManager* (`BATTERY_PROPERTY_CURRENT_NOW`) and voltage via *ACTION_BATTERY_CHANGED* (`EXTRA_VOLTAGE`), sampled periodically during *idle* and *load* (benchmark) windows. Power was computed as $P = V \cdot I$ and incremental power as $\Delta P = P_{load} - P_{idle}$, enabling energy per inference as ΔP/throughput from 200 power samples per device.

For the ESP32 (Espressif microcontroller), inference ran in C/C++ (ESP-IDF) with a dedicated benchmark routine. We performed 10 *warmup runs*, 1000 executions to estimate mean latency and

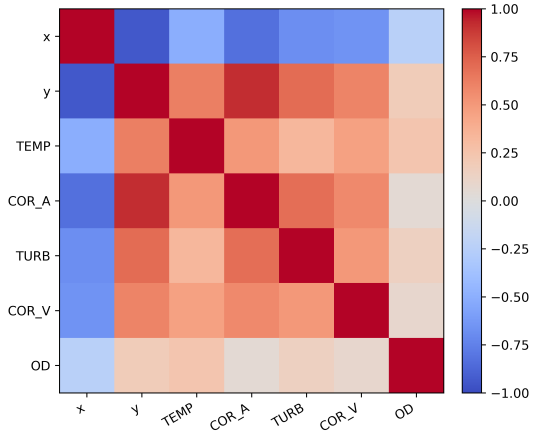
throughput, and 200 individual samples for standard deviation, as reported in the firmware serial log.

4 RESULTS

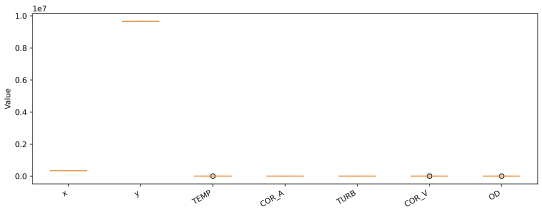
4.1 Exploratory analysis

The results below correspond to the SNN model tuned with data augmentation (light Gaussian noise) and hyperparameter selection via grid search. We first report exploratory analysis to characterize variable distributions and relationships before presenting model performance, spatial results, and energy/latency benchmarks.

In Figure 3(a), the correlation matrix reveals relevant dependencies among physicochemical parameters and contextual variables, indicating that multivariate prediction is appropriate for the problem. Higher correlations suggest that part of target variability can be explained by combinations of temperature, apparent color, turbidity, and OD, while lower correlations point to local variability components and nonlinearity. In Figure 3(b), the boxplots show heterogeneous dispersions and the presence of extreme values, justifying the use of normalization, data augmentation, and regularization to avoid excessive sensitivity to isolated samples.



(a) Variable correlations



(b) Variable boxplots

Figure 3: *Exploratory analysis: correlations and distribution of input variables.*

In Figure 4, the relatively narrow pH range indicates that small absolute changes are environmentally meaningful; therefore, modest average errors can still be relevant in practice.

4.2 Training dynamics

Figure 5 reports the optimization trajectory for the selected multimodal SNN. The curve shows a rapid initial loss decrease followed by a stable plateau with low train/validation divergence, which supports convergence under the adopted regularization strategy and justifies *early stopping*. This result is presented as evidence of training stability, while the effect of synthetic data augmentation is assessed separately in the dedicated augmentation analyses and ablation subsection.

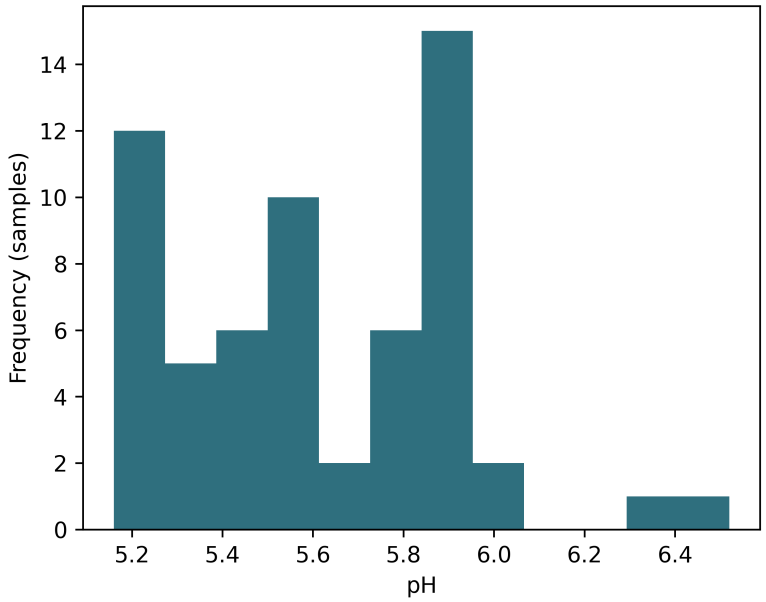


Figure 4: *Distribution of the pH target in the field dataset.*

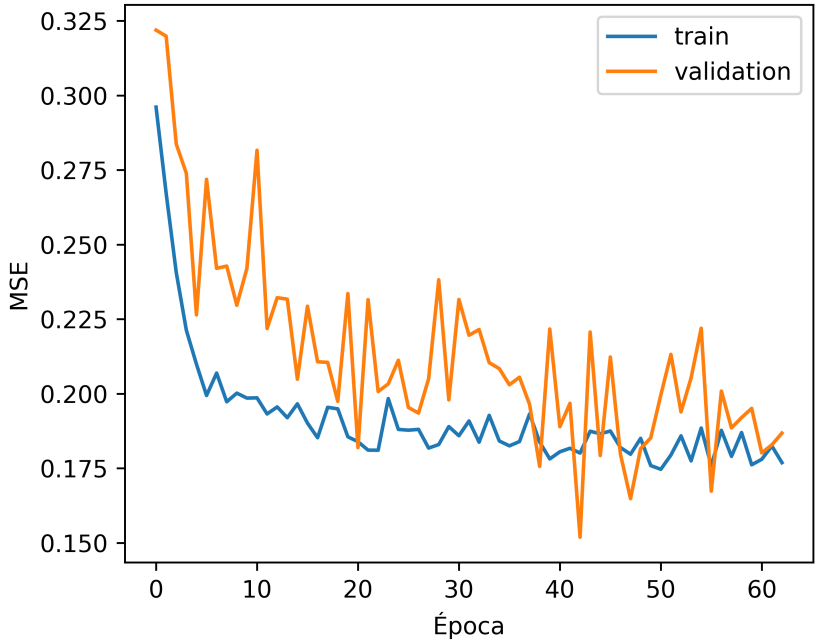


Figure 5: *Training and validation loss (MSE) across epochs for the selected multimodal SNN.*

4.3 Data augmentation

Figure 6 shows strong overlap between original and augmented data, indicating that Gaussian noise densified the sample without shifting means or creating spurious modes. This preserves

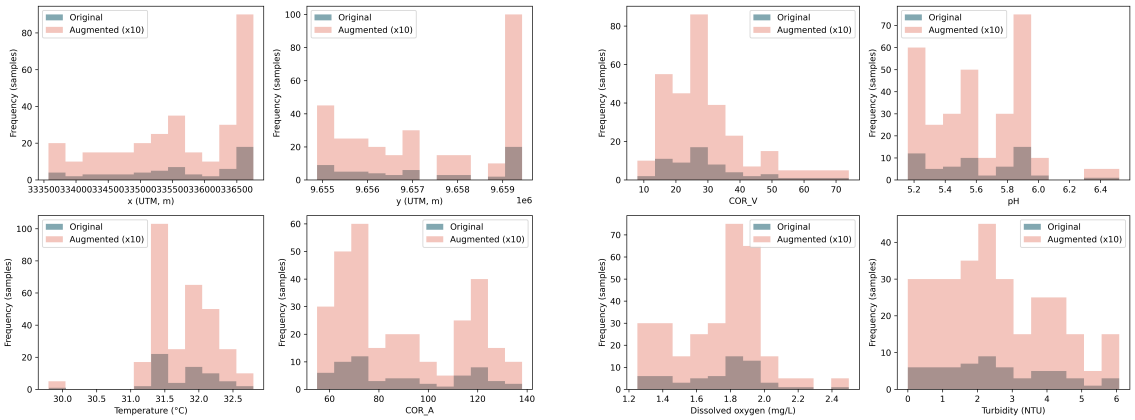


Figure 6: *Original vs. augmented data distributions (x10, Gaussian noise): histograms.*

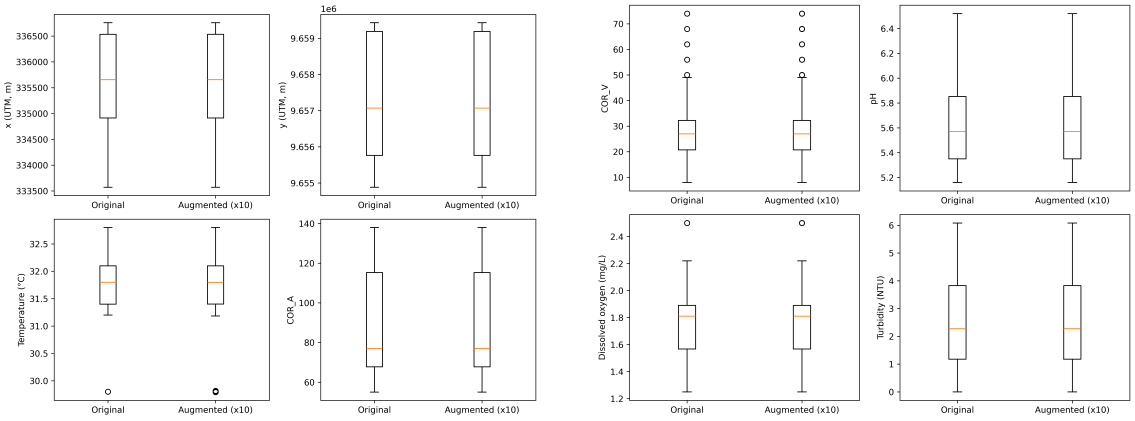


Figure 7: *Original vs. augmented data distributions (x10, Gaussian noise): boxplots.*

physical coherence and acts as a regularizer for a small dataset.

Figure 7 reinforces this interpretation: medians and quartiles remain close, with a slight increase in dispersion and no extreme outliers beyond the original range. This indicates controlled variability, reducing sensitivity to measurement noise and favoring robustness.

4.4 Augmentation ablation

To quantify the effect of synthetic augmentation on generalization, we ran an ablation with the SNN under the same protocol and hyperparameters, comparing *no augmentation* (factor 1) versus *augmentation x10* (Gaussian noise, $\sigma = 0{,}01$) across 5 seeds $(7, 13, 21, 42, 84)$. Augmentation reduced macro MAE from $0{,}496 \pm 0{,}060$ to $0{,}483 \pm 0{,}075$, corresponding to a 2,53% reduction in macro MAE. Macro R^2 increased slightly ($0{,}254 \rightarrow 0{,}256$), while macro RMSE remained essentially stable ($0{,}615 \rightarrow 0{,}617$). These results support augmentation as a stabilization strategy for the small dataset, especially by reducing turbidity MAE in this ablation ($1{,}139 \rightarrow 1{,}102$).

Figure 8 provides a visual summary of this ablation and helps clarify the practical effect of augmentation on model behavior: the main benefit is a reduction in macro MAE, which is

consistent with improved average absolute prediction error under repeated splits, while the macro RMSE remains nearly unchanged and the macro R^2 shows only a slight increase, indicating that augmentation in this setting acts primarily as a training stabilizer for a small dataset rather than as a transformation that fundamentally changes the global fit of the model; in particular, the combined pattern is compatible with a modest but meaningful improvement in generalization, especially for the most difficult target (turbidity), without evidence of a broad distortion of the error profile across outputs.

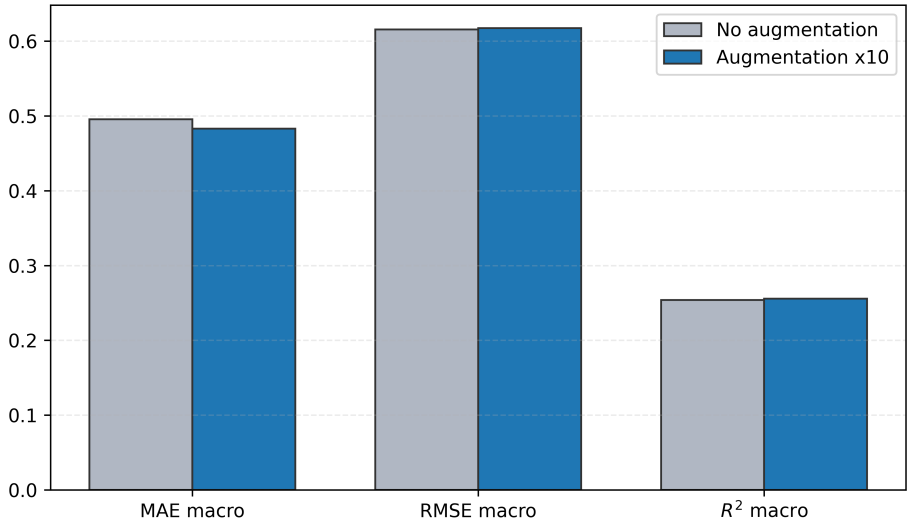


Figure 8: *Augmentation ablation (5 seeds): no augmentation vs augmentation x10 for macro metrics.*

4.5 Model performance and comparisons

Table 5 summarizes the five best multimodal grid-search combinations, ranked by lowest validation loss.

Table 5: *Top-5 grid-search configurations (ranked by lowest validation loss).*

Hidden	T	lr	β	$thresh$	σ	val_loss
64	15	0,002	0,95	0,8	0,01	0,5998
64	25	0,001	0,9	1,0	0,01	0,6095
64	25	0,001	0,9	1,0	0,02	0,6097
64	15	0,002	0,95	0,8	0,02	0,6108
96	25	0,002	0,9	0,8	0,03	0,6117

A multimodal (multi-output) model was trained to jointly predict pH, dissolved oxygen, and turbidity. The best multimodal model achieved test metrics of $MAE_{pH} = 0{,}107$, $MAE_{OD} = 0{,}222$, and $MAE_{TURB} = 0{,}971$. Without leakage between inputs and targets, turbidity remained the hardest variable, indicating a more challenging and realistic prediction task.

To assess robustness, we ran a multi-seed benchmark ($n = 10$: 7, 13, 21, 42, 84, 101, 123, 202, 303, 404), comparing the multimodal SNN with Random Forest (RF), Multilayer Perceptron (MLP), and multi-output LSTM using the same seed-based split. In terms of global (macro) error, Random Forest and LSTM achieved lower average MAE and $RMSE$, while the SNN remained competitive. The MLP showed higher instability across seeds. Table 6 summarizes mean and standard deviation values. All reported R^2 values fall within the mathematically valid range ($R^2 \leq 1$). In the paired sign test (SNN vs Random Forest, $n = 10$ seeds), we obtained $p = 0{,}3438$ for MAE_{macro}, $p = 1{,}0000$ for $RMSE_{macro}$, and $p = 1{,}0000$ for R^2_{macro}. Thus, for $\alpha = 0{,}05$, there is no statistical evidence of global superiority of the SNN over Random Forest on this dataset. When compared to MLP under the same protocol, the SNN showed a clear advantage in average performance and stability. The sign test yielded $p = 0{,}0020$ for MAE_{macro}, $RMSE_{macro}$, and R^2_{macro} (SNN vs MLP), indicating MLP sensitivity to small data. In addition, the MLP R^2_{macro} was negative on average ($-5{,}460 \pm 5{,}220$), indicating low robustness in this setting.

Table 6: *Multi-seed comparison (mean ± standard deviation) among multimodal models.*

Model	MAE macro	RMSE macro	R^2 macro
Random Forest	$\mathbf{0{,}409 \pm 0{,}062}$	$\mathbf{0{,}557 \pm 0{,}086}$	$\mathbf{0{,}335 \pm 0{,}131}$
LSTM	$0{,}428 \pm 0{,}059$	$0{,}548 \pm 0{,}073$	$0{,}313 \pm 0{,}139$
SNN	$0{,}447 \pm 0{,}064$	$0{,}566 \pm 0{,}072$	$0{,}291 \pm 0{,}146$
MLP	$0{,}781 \pm 0{,}193$	$0{,}983 \pm 0{,}178$	$-5{,}460 \pm 5{,}220$

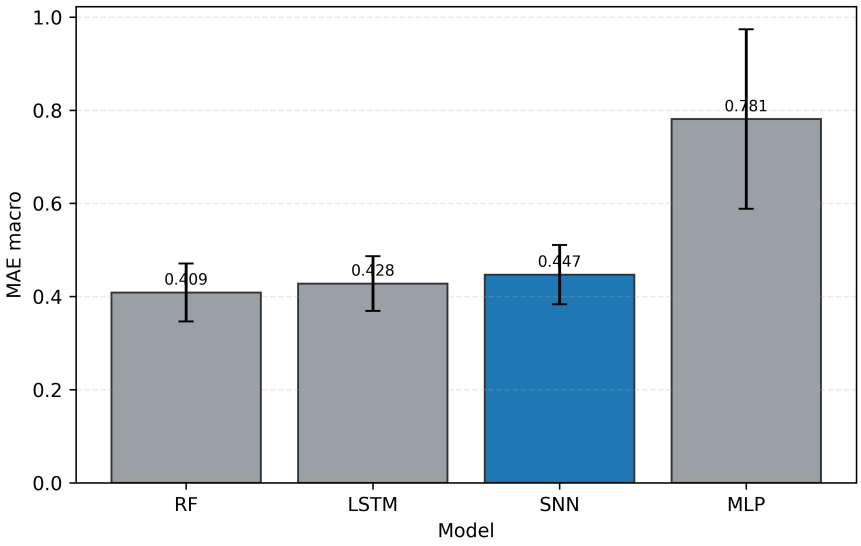


Figure 9: *Comparative macro MAE across multimodal models (mean with standard deviation bars).*

Figure 9 provides a compact visual comparison of the macro MAE for all evaluated multimodal models, making the ranking easier to interpret than the table alone by simultaneously showing central tendency and variability (standard deviation bars), and highlighting that Random Forest and LSTM achieve lower mean error while the SNN remains in the same performance band with

substantially better behavior than the MLP in this small-data setting.

In the SNN vs LSTM comparison, the sign test yielded $p = 0{,}3438$ for MAE_{macro}, $p = 0{,}3438$ for $RMSE_{macro}$, and $p = 0{,}1094$ for R^2_{macro}, with no statistically significant difference at $\alpha = 0{,}05$. Focusing on turbidity, we performed a loss-weight sweep in the SNN, prioritizing TURB. The best compromise was achieved with weights $(0{,}05,\ 0{,}15,\ 0{,}8)$ for (pH, OD, TURB), reducing mean MAE_{TURB} to 1,006 (10 seeds) relative to the base configuration $(0{,}2,\ 0{,}2,\ 0{,}6)$, which yielded $MAE_{TURB} = 1{,}012$. This adjustment improves the most difficult target, with a slight decrease in R^2_{macro} and marginal change in MAE_{macro}, indicating a trade-off between turbidity focus and global performance.

In summary, Table 7 shows the loss-weight sweep and highlights the trade-off between reducing turbidity error and preserving global performance. When the TURB weight increases to 0.80, MAE_{TURB} decreases to 1,006, but R^2_{macro} drops to 0,280, indicating weaker overall fit. The base configuration $(0{,}20, 0{,}20, 0{,}60)$ provides a better balance across variables, with slightly lower MAE_{macro} and higher R^2_{macro}.

Table 7: *SNN loss-weight sweep (10 seeds).*

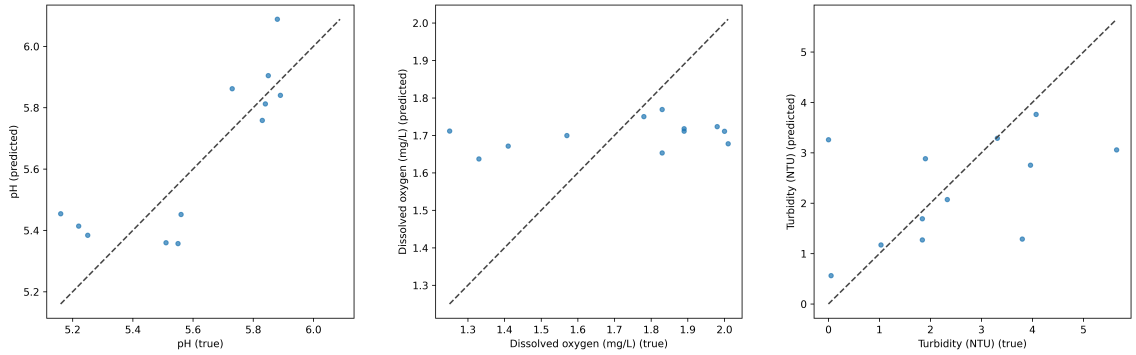
Weights (pH, OD, TURB)	MAE macro	MAE TURB	R^2 macro
$(0{,}20,\ 0{,}20,\ 0{,}60)$	**0,447**	1,012	**0,291**
$(0{,}15,\ 0{,}15,\ 0{,}70)$	0,452	1,022	0,289
$(0{,}10,\ 0{,}10,\ 0{,}80)$	0,454	1,028	0,288
$(0{,}05,\ 0{,}15,\ 0{,}80)$	0,448	**1,006**	0,280

In Figure 10(a) and (b), the point clouds for pH and OD align more closely to the identity line, reflecting better fit for these targets. In Figure 10(c), higher dispersion for turbidity indicates additional modeling difficulty, consistent with the intrinsic variability of this parameter and possible local effects not fully captured by the inputs. Even so, the global trend remains increasing, indicating that the model preserves predictable structure for turbidity as well.

The observed behavior suggests that pH and OD are more directly explained by the available inputs, while turbidity may depend on unmeasured local factors such as rainfall events, resuspension, or point-source inputs. Even with higher error, the model maintains a monotonic relationship with true turbidity, indicating potential improvement with new variables and denser data.

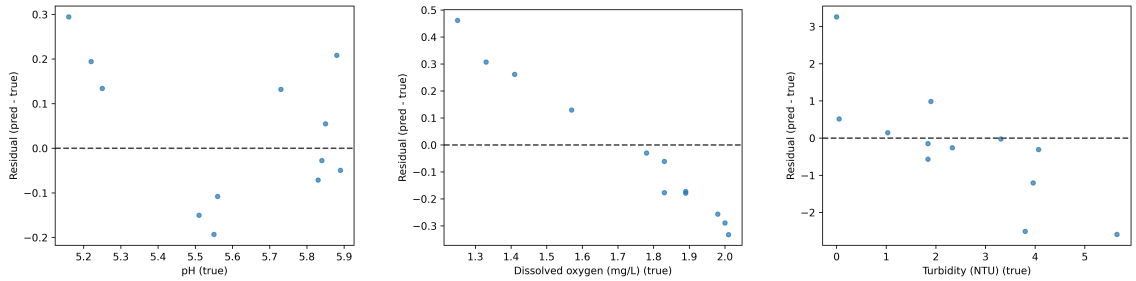
In Figure 11(a) and (b), residuals for pH and OD oscillate around zero with no strong directional pattern, suggesting the absence of dominant systematic bias. In Figure 11(c), residual amplitude for turbidity is larger and less homogeneous, indicating higher uncertainty for this component. This evidence motivated the loss-weight tuning stage, prioritizing turbidity to reduce specific error without substantial loss in overall performance.

The residual distribution also indicates that extreme errors are more common for turbidity, which corroborates the interpretation of the predicted-vs-true plots. By prioritizing turbidity in the loss function, we sought to reduce the impact of these larger errors without compromising pH and OD. This trade-off is expected in multi-output models and should be explicit for transparency in practical applications.



(a) pH *(b) OD* *(c) Turbidity*

Figure 10: *Predicted vs. true values for the multimodal model (pH, dissolved oxygen, turbidity).*



(a) pH *(b) OD* *(c) Turbidez*

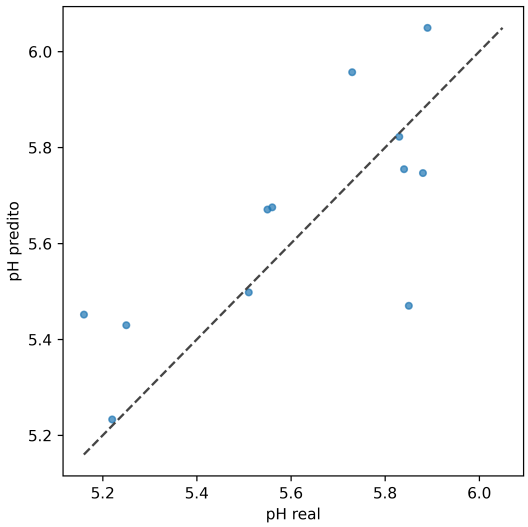
Figure 11: *Residuals (predicted - true) of the multimodal model for each variable.*

4.6 Spatial results and pollution index

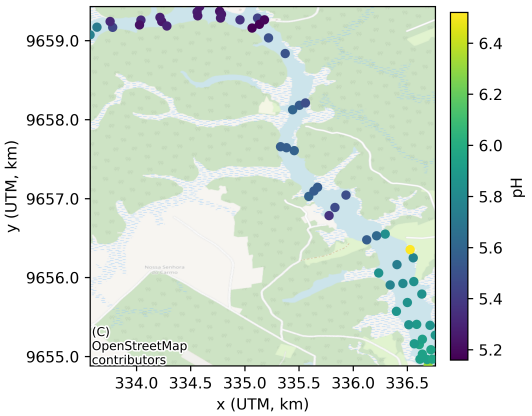
In Figure 12(a), the concentration of points near the identity line indicates good ability to reproduce observed pH, with residual errors more evident at range extremes where sample density is lower. This pattern is expected in small datasets and reinforces the need to increase sampling to reduce uncertainty in underrepresented regions. In Figure 12(b), the pH spatial map shows a consistent geographic gradient, indicating that the model preserves spatial coherence rather than producing random predictions across the lake.

Beyond global fit, the predicted-vs-true plot suggests that the largest discrepancies occur at extreme values, where the model has fewer examples to learn the pattern. On the map, spatial continuity of pH indicates that predictions respect the lake's geographic structure, which is important for micro-zone analysis and prioritizing inspection areas.

In Figure 13(a), dissolved oxygen exhibits spatial variation consistent with heterogeneous aquatic environments, with low-oxygen zones potentially more sensitive to environmental stress. In Figure 13(b), turbidity shows stronger contrasts, indicating regions with higher particulate load. Interpreting OD and turbidity together is important because it combines a biochemical indicator with a physical indicator of suspended material, strengthening environmental diagnosis through multiple lines of evidence.



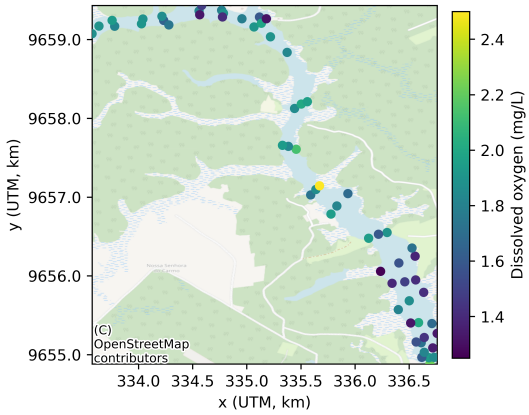
(a) Predicted vs. true



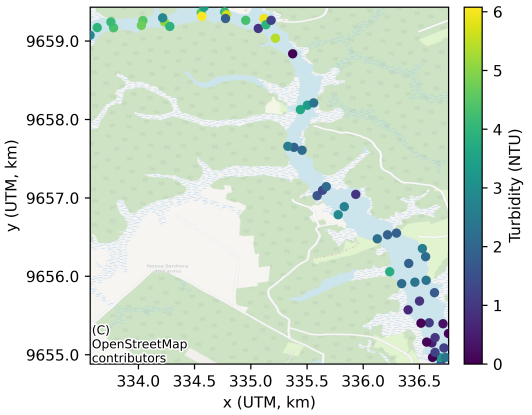
(b) Spatial map of pH

Figure 12: *Predictive performance and spatial pattern of pH in Lago do Serpa.*

Interpretively, low OD regions may indicate higher biological oxygen demand or reduced water renewal, while high turbidity is often associated with sediments, suspended organic matter, or external inputs. When both variables point to degradation in the same region, confidence in identifying environmental hotspots increases, even with a limited sample set.



(a) Spatial map of dissolved oxygen



(b) Spatial map of turbidity

Figure 13: *Spatial patterns of dissolved oxygen and turbidity.*

Figure 14 transforms point measurements into continuous surfaces via IDW, enabling the visualization of "patches" and spatial transitions for each variable. Although interpolation does not replace direct measurement, it improves integrated reading of the system by highlighting high-contrast zones and areas prioritized for future sampling. The coherence between estimated

surfaces and point patterns reinforces the dataset's internal consistency.

Interpolated surfaces also help communicate results to managers and field teams because they provide a continuous view of the lake without requiring point-by-point interpretation. Given the limited sampling density, these surfaces should be understood as exploratory approximations rather than definitive maps, but they are still useful for guiding future campaigns and validating hypotheses about spatial dynamics.

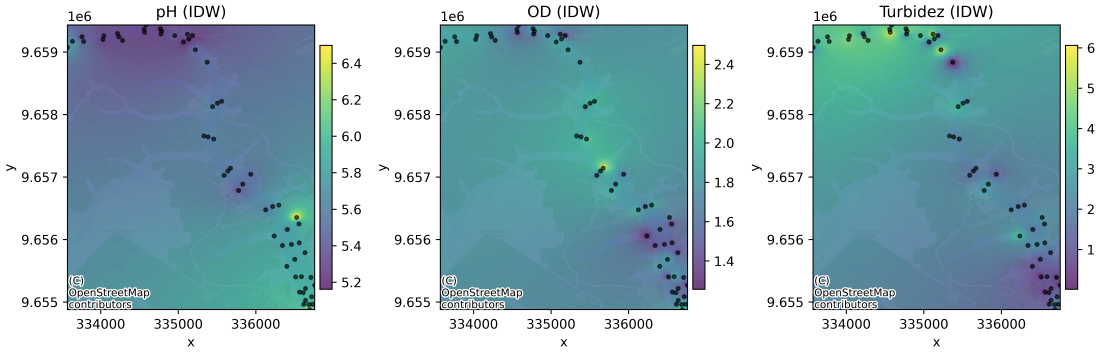
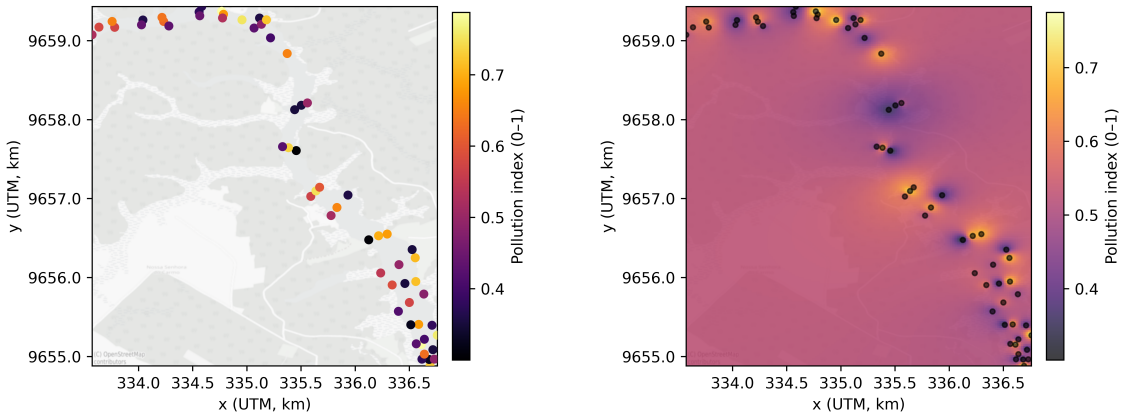


Figure 14: *IDW interpolated surfaces for pH, dissolved oxygen, and turbidity from sampled points.*

In Figure 15(a), the composite index spatially orders points by relative degradation intensity, integrating pH, OD, and turbidity into a single scale. In this work, the index is normalized to the $[0, 1]$ range, where higher values indicate worse relative condition in the analyzed set. In Figure 15(b), the IDW surface extends this reading by delineating the estimated pollution patch, useful for prioritizing field inspections. This representation does not replace laboratory analysis, but functions as an operational screening tool for continuous monitoring and decision support.

The composite index synthesizes multiple signals into a single operational indicator, facilitating communication with non-specialist users and prioritization of critical areas. The IDW surface shows that the estimated patch is not uniform, suggesting spatial patterns that can guide new sampling or interventions. This type of visualization is particularly useful in edge contexts, where fast and interpretable decisions are essential.



(a) Pollution index at sample points

(b) IDW surface of the index

Figure 15: *Spatial estimate of the composite pollution index from multimodal predictions (normalized scale* $[0, 1]$*) over an OpenStreetMap basemap (UTM 21S).*

4.7 Latency and energy

The final TFLite model was executed on a Raspberry Pi 4B (ARM64) with XNNPACK enabled. Mean inference latency was 0,214 ms/sample (standard deviation 0,023 ms), with throughput of 4670,50 samples/s (Table 8). These results confirm the feasibility of local execution on low-power hardware with very low per-sample cost. In practical terms, inference is sub-millisecond per sample and supports thousands of predictions per second, compatible with near-real-time monitoring.

Table 8: *TFLite inference latency on Raspberry Pi 4B.*

Device	Mean latency (ms)	Std. dev. (ms)	Throughput (samples/s)
Raspberry Pi 4B	0,214	0,023	4670,50

On Android Device A (Table 4), the in-app benchmark reported mean latency of 0,093 ms/sample (std. dev. 0,005 ms) and throughput of 10780,37 samples/s over 5000 runs. These values indicate that local inference is feasible in near-real-time even without dedicated acceleration. Latency remains below 1 ms per sample and throughput exceeds 10k samples/s, sufficient for field use with frequent updates. Table 9 organizes these values for comparison with the Raspberry Pi.

Table 9: *Inference latency on Android devices.*

Device	Mean latency (ms)	Std. dev. (ms)	Throughput (samples/s)
Device A	0,093	0,005	10780,37
Device B	0,030	0,002	33425,81
Device C	0,61	0,014	16313,16

Beyond latency, incremental energy was estimated directly on the phones via system current and voltage. For Device A, mean idle power was 1,977 W and load power 2,106 W, yielding $\Delta P = 0{,}128$ W and 0,0104 mJ/inf (12337,40 samples/s). For Device B, mean idle power was 3,455 W and load power 3,923 W, yielding $\Delta P = 0{,}468$ W and 0,0151 mJ/inf (30905,08 samples/s). For Device C, mean idle power was 8,158 W and load power 8,204 W, yielding $\Delta P = 0{,}046$ W and 0,00285 mJ/inf (16121,71 samples/s). In all cases, 200 power samples were collected (Table 10).

Table 10: *Incremental energy per inference on Android devices.*

Device	P_{idle} (W)	P_{load} (W)	ΔP (W)	Energy (mJ/inf)	Samples
Device A	1,977	2,106	0,128	0,0104	200
Device B	3,455	3,923	0,468	0,0151	200
Device C	8,158	8,204	0,046	0,00285	200

On the ESP32 running MicroPython, the benchmark reported mean latency of 199,298 ms (std. dev. 7,535 ms) and throughput of 5,0 samples/s over 500 runs, indicating multimodal inference feasibility on a microcontroller with fully offline execution (Table 11).

Table 11: *ESP32 inference benchmark.*

Metric	Value	Unit
Mean latency	199,298	ms
Std. dev.	7,535	ms
Throughput	5,0	samples/s
Total spikes	136 (14,2%)	–
Free memory	134,544	bytes

For ESP32 energy (MicroPython inference), mean idle power was 0,189 W and load power 0,245 W, yielding $\Delta P = 0{,}056$ W. With throughput of 5,0 samples/s, estimated energy per inference was 11,204 mJ/inf. These values were obtained from 50 idle samples and 50 load samples (Table 12). Sensor verification was based on the stability of idle and load windows and on consistent EMA-smoothed readings.

Table 12: *Incremental energy per inference (ESP32).*

P_{idle} (W)	P_{load} (W)	ΔP (W)	Energy (mJ/inf)	N_{idle}/N_{load}
0,189	0,245	0,056	11,204	50/50

For the ESP32 running the MicroPython SNN, 50 idle samples yielded mean $V = 4{,}873$ V, $I = 38{,}906$ mA, and $P = 189{,}426$ mW (std: 0,0059 V, 0,169 mA, 0,448 mW). Under load (50 samples during inference), mean values were $V = 4{,}877$ V, $I = 51{,}538$ mA, and $P = 245{,}447$ mW (std: 0,0059 V, 2,822 mA, 15,820 mW). With throughput of 5,0 samples/s, the incremental power was $\Delta P = 56{,}021$ mW, yielding 11,204 mJ/inf for the MicroPython setup.

From a comparative perspective (Table 13), the ESP32 shows the highest energy per inference, which is expected given its lower processing rate (throughput) and the absence of dedicated vector

acceleration. Even so, the value remains viable for intermittent sampling scenarios typical of low-rate sensor networks. The Raspberry Pi 4B and smartphones present lower mJ/inf values, reflecting higher computational capacity and CPU/system-level optimizations. Device B, for example, combines high throughput with moderate incremental power, resulting in low energy per inference.

Table 13: *Comparative summary of energy per inference across platforms.*

Platform	ΔP (W)	Energy (mJ/inf)	Throughput (samples/s)
ESP32 (MicroPython)	0,056	11,204	5,0
Raspberry Pi 4B	0,239	0,0512	4670,50
Device A (Android)	0,128	0,0104	12337,40
Device B (Android)	0,468	0,0151	30905,08
Device C (Android)	0,046	0,00285	16121,71

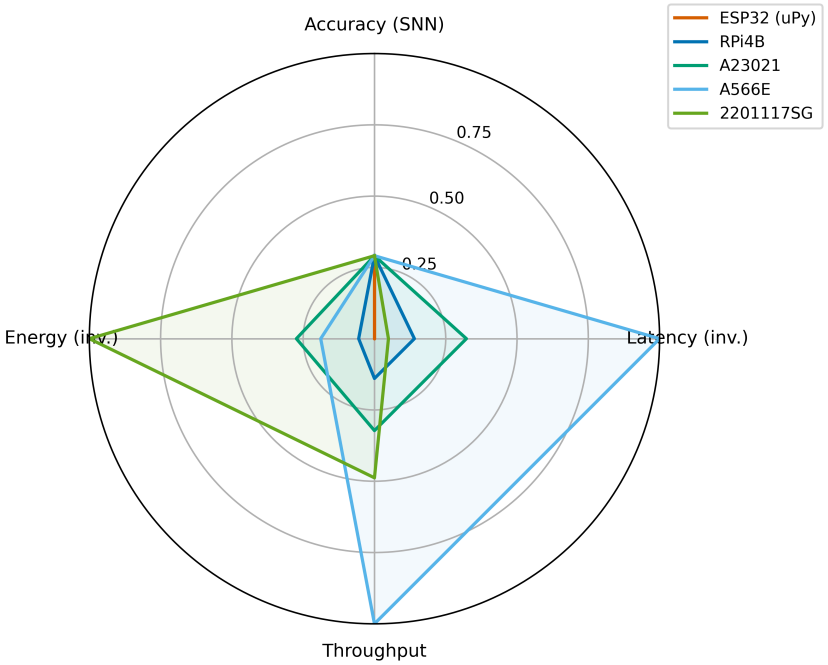


Figure 16: *SNN deployment comparison across platforms (radar chart, normalized metrics). Energy and latency measurements are available only for deployment platforms, not for all baseline models.*

Figure 17 complements Table 13 by presenting the energy-per-inference comparison on a logarithmic scale, which is necessary to visualize, in a single plot, the large gap between the ESP32

MicroPython implementation and the Raspberry/Android devices without visually compressing the lower-consumption platforms into an indistinguishable range.

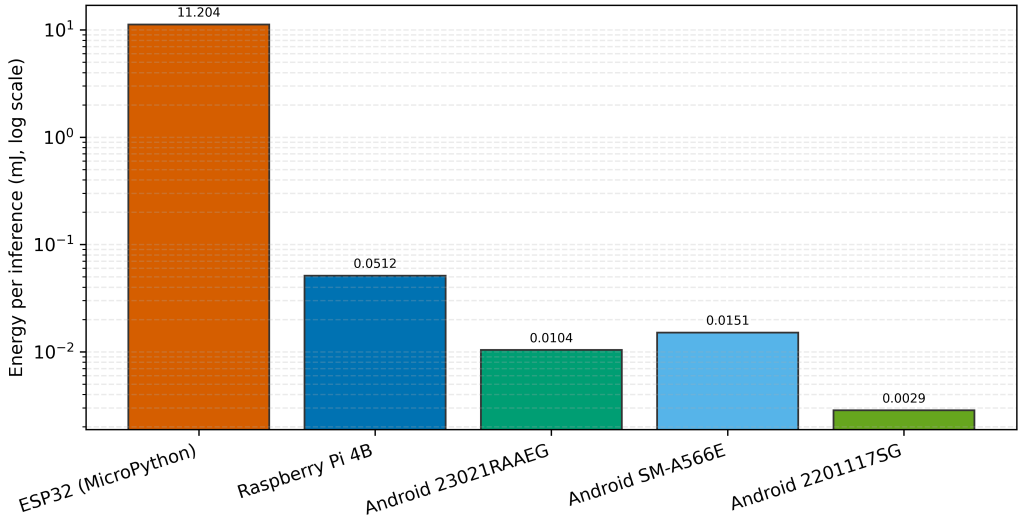


Figure 17: *Comparative energy per inference across platforms (log scale).*

Figure 18 adds the joint processing-speed/energy perspective and makes the platform trade-off explicit: moving to the right means faster inference and moving downward means lower incremental energy per inference. In this view, Device B is the fastest platform but does not have the minimum mJ/inf, while Device C combines high throughput with the lowest measured energy per inference; ESP32 remains in the opposite corner, with much lower throughput and higher mJ/inf, which is consistent with its microcontroller constraints and with its intended use in low-rate autonomous sampling rather than high-volume inference workloads.

Figure 16 summarizes the deployment trade-offs for the SNN across platforms using normalized metrics (accuracy reference, inverse latency, throughput, and inverse energy), which helps communicate that no single platform dominates all criteria simultaneously: smartphones and Raspberry Pi concentrate strong throughput/latency efficiency, whereas ESP32 remains operationally relevant for low-rate offline scenarios despite its weaker efficiency envelope.

Device C showed higher energy per inference despite high throughput, suggesting larger incremental system power during the benchmark. This highlights that, on mobile devices, consumption can vary significantly across models and system configurations, reinforcing the importance of device-specific empirical measurements when energy efficiency is a design requirement.

To estimate incremental inference energy on the Raspberry Pi 4B, an INA219 sensor was placed in series with the supply, measuring current in idle state and under load (benchmark). Incremental consumption was computed as $\Delta I = I_{load} - I_{idle}$, and incremental power as $\Delta P = V \cdot \Delta I$, with $V = 5{,}0$ V. Averages were computed from $N_{idle} = 97$ samples and $N_{load} = 440$ samples. With mean idle current of 0,548 A and load current of 0,595 A, we obtained $\Delta P = 0{,}239$ W. Using the observed throughput (4670,50 samples/s), the estimated energy per inference was 0,0512 mJ/inf (Table 14). These values provide a practical estimate of the additional energy cost of the model on embedded hardware.

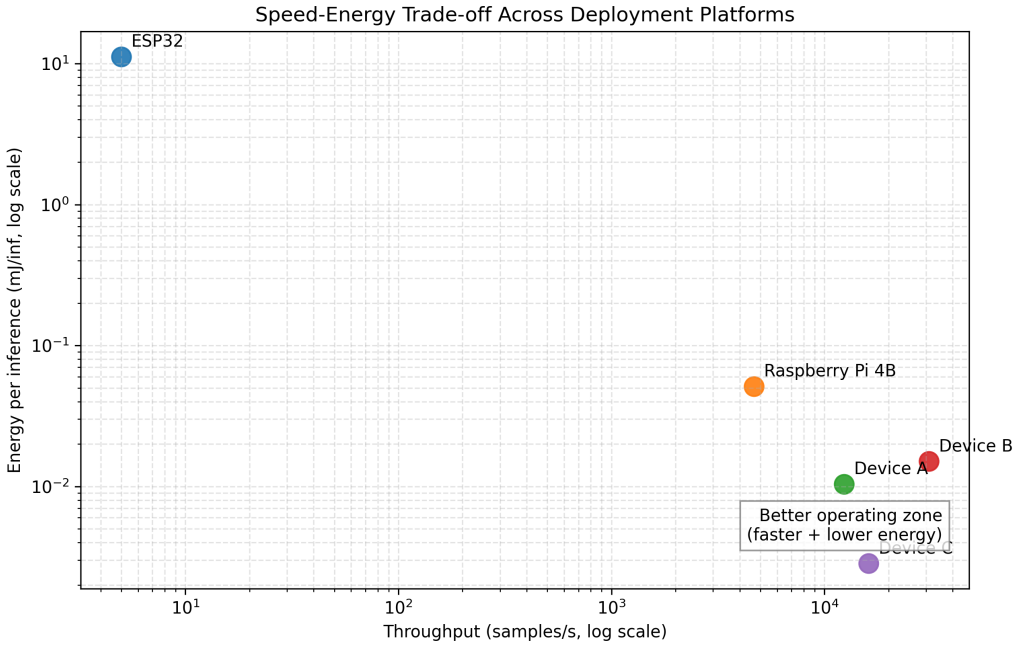


Figure 18: *Speed-energy trade-off across deployment platforms (throughput vs. energy per inference, log scales).*

Table 14: *Incremental energy per inference (Raspberry Pi 4B).*

I_{idle} (A)	I_{carga} (A)	N_{idle}/N_{carga}	ΔP (W)	Energia (mJ/inf)
0,548	0,595	97/440	0,239	0,0512

5 DISCUSSION

The results indicate that the proposed neuromorphic approach is viable for predicting physicochemical water parameters in the Amazonian context, with consistent performance after hyperparameter tuning and *early stopping* regularization. In the multimodal scenario without information leakage (i.e., target variables are not used as inputs), prediction difficulty increases, especially for turbidity, which makes the evaluation more realistic for field use.

In the 10-seed benchmark, Random Forest and LSTM achieved lower global mean errors, while the SNN remained competitive and clearly more robust than the MLP. Statistical tests reinforce this reading: there was no evidence of statistical superiority of the SNN over Random Forest and LSTM at $\alpha = 0{,}05$, but there was a significant advantage of the SNN over MLP. Therefore, the results support SNN as a viable edge alternative, even if it is not dominant in absolute error on the current dataset. Spatially, the pH, OD, and turbidity maps reveal coherent gradients and clusters, reinforcing the consistency of field data and suggesting that input variables carry relevant information for prediction. The correlations between temperature, apparent color, and turbidity observed in the exploratory analysis align with expected physicochemical behavior reported for lentic systems in the water-quality literature, particularly regarding the relevance of thermal regime and oxygen dynamics, and the recurring use of physicochemical predictors in data-driven

monitoring studies (JANE et al., 2021; YAN et al., 2024); moreover, the adoption of pH, dissolved oxygen, and turbidity as core response variables is consistent with regulatory and monitoring practice in Brazilian inland waters (RESOLUçãO. . . , 2005), providing additional support for the proposed model and for the environmental interpretability of the predicted outputs without claiming site-specific causality beyond the observed dataset. Although Random Forest achieved lower macro error on this dataset, it does not provide event-driven temporal processing or a direct path to neuromorphic deployment, which keeps the SNN relevant for low-power adaptive edge scenarios where hardware compatibility and temporal sparse computation are part of the design objective rather than only predictive accuracy.

Beyond predictive performance, model execution on the Raspberry Pi 4B achieved sub-millisecond latency per sample, reinforcing the feasibility of local inference on low-power hardware compatible with remote and offline scenarios. Validation on the ESP32 extends this result to microcontrollers: despite lower throughput, the benchmark indicated millisecond-level latency and consistent absolute error, demonstrating that the multimodal model can operate autonomously on highly constrained devices.

From a practical standpoint, differences among models suggest that the choice depends on the operational objective. Random Forest and LSTM tend to minimize mean error, while the SNN offers a robust alternative when local inference with lower computational cost and potential energy efficiency is prioritized. The more unstable behavior of the MLP confirms that dense classical models are sensitive on small datasets and can vary significantly across seeds. Additionally, systematic difficulty in turbidity indicates that this parameter requires additional information (e.g., hydrodynamic or seasonal variables) to improve generalization. Although the original field dataset comprises 60 samples, synthetic augmentation and repeated seed-based splits were used to reduce variance and sensitivity to overfitting; however, these strategies do not replace broader field campaigns and external validation across additional hydrological contexts.

In energy terms, the comparison among ESP32, Raspberry Pi, and Android devices shows that incremental energy per inference varies significantly with hardware and available throughput. The ESP32 yielded higher energy per inference but remains suitable for low-rate sampling applications, while Raspberry Pi and smartphones offer better per-inference efficiency in scenarios requiring higher prediction volumes. These results confirm that platform choice should consider not only accuracy, but also sampling regime and energy constraints in the deployment environment.

Another important consideration is that temporal encoding via *rate coding* implies a trade-off between temporal fidelity and computational cost: larger T values can improve representation quality, but increase inference latency and energy use. This suggests that, in real-world deployments, T should be calibrated according to operational context and energy constraints of the edge device.

6 CONCLUSION

This work confirmed the technical feasibility of neuromorphic prediction of water quality in a remote Amazonian region by combining multimodal modeling, spatial analysis, and embedded deployment. In terms of performance, the SNN achieved competitive and stable results, outperforming the MLP in the multi-seed benchmark and remaining close to classical baselines. In the evaluated dataset, Random Forest and LSTM achieved lower global mean absolute error,

while the SNN remained a robust alternative for neuromorphic edge inference. In addition, loss-weight tuning produced a specific improvement in turbidity with a moderate trade-off in macro performance.

On the application axis, the model was exported to TensorFlow Lite and integrated into an offline mobile app, enabling local inference of pH, OD, and turbidity, composite pollution index calculation, and geospatial visualization with an offline map (MBTiles). This result advances the work beyond a purely algorithmic proof of concept by demonstrating a full pipeline of data, training, embedded inference, and field decision support.

In neuromorphic design, *rate coding* was also coherent with the edge scenario, as it allows explicit control of the balance between temporal representation quality and mobile inference latency via the parameter T. As limitations, the study still relies on a local and reduced sample base, which restricts generalization to other water bodies and seasonal regimes; synthetic data augmentation does not replace real-world collection, and model differences may vary with new field campaigns. Practical validation was carried out on Android, Raspberry Pi, and ESP32, but extrapolation to other microcontrollers and different energy profiles requires additional measurements and broader field campaigns. As future continuity, it is recommended to expand data collection, validate the model across multiple environmental scenarios, and systematically measure latency, energy consumption, and operational robustness on different edge devices. It is also relevant to refine the calibration of the composite pollution index with support from environmental specialists and to test field model-update strategies under different hydrological regimes.

As an additional contribution, ESP32 validation showed that multimodal inference is feasible on microcontrollers, with millisecond-level latency and absolute errors compatible with the mobile scenario. The energy comparison among ESP32, Raspberry Pi, and smartphones revealed clear trade-offs between throughput and energy per inference, reinforcing the importance of selecting platform and sampling regime according to operational context. Thus, the core differentiator of this work is not only the use of SNNs, but the delivery of a complete and measurable offline inference pipeline: data $\rightarrow$ model $\rightarrow$ app $\rightarrow$ map $\rightarrow$ energy. This study shows that neuromorphic inference for water-quality monitoring is not only conceptually viable, but also deployable and measurable under real offline, hardware, and energy constraints.

ACKNOWLEDGMENTS

The authors thank the Automation, Computer Vision, Artificial Intelligence, and Robotics Laboratory (NAVIR), the Graduate Program in Science, Technology and Health (PPGCTS), the Federal University of Amazonas (UFAM), Motorola Mobility Comércio de Produtos Eletrônicos Ltda and Flextronics da Amazônia Ltda, as provided for in Arts. 21 and 22 of Decree No. 10.521/2020, under Federal Law No. 8.387/1991 and agreement No. 008/2024, signed with ICET/UFAM, through the Research, Development, and Technological Innovation Project (PDI) “Mob4Ai - Advanced Artificial Intelligence for neuromorphic mobile system”.

References

AHMED, A. N. et al. Machine learning methods for better water quality prediction.

Journal of Hydrology, Amsterdã, v. 578, p. 124084, nov. 2019. Disponível em: <https://doi.org/10.1016/j.jhydrol.2019.124084>.

AL-ADHAILEH, M. H.; ALSAADE, F. W. Modelling and prediction of water quality by using artificial intelligence. *Sustainability*, Basileia, v. 13, n. 8, p. 4259, abr. 2021. Disponível em: <https://doi.org/10.3390/su13084259>.

AL-SHAREEDA, M. A. et al. Secure iot-based real-time water level monitoring system using esp32 for critical infrastructure. *Journal of Cyber Security and Risk Auditing*, v. 2025, n. 2, p. 44–52, 2025. Disponível em: <https://doi.org/10.63180/jcsra.thestap.2025.2.4>.

ALPROL, A. E. et al. Artificial intelligence technologies revolutionizing wastewater treatment: Current trends and future prospective. *Water*, Basileia, v. 16, n. 2, p. 314, jan. 2024. Disponível em: <https://doi.org/10.3390/w16020314>.

ARAL, A. The promise of neuromorphic edge ai for rural environmental monitoring. *Environmental Data Science*, v. 3, p. e34, 2024. Disponível em: <https://doi.org/10.1017/eds.2024.36>.

AZADI, B. et al. Towards an energy-efficient and sustainable iiot using embedded neuromorphic ai. In: *Proceedings of the 15th International Conference on the Internet of Things*. New York, NY, USA: Association for Computing Machinery, 2025. (IOT '25), p. 237–244. Disponível em: <https://doi.org/10.1145/3770501.3770529>.

CHADALAVADA, S. et al. Gated-lnn: Gated liquid neural networks for accurate water quality index prediction and classification. *IEEE Access*, v. 13, p. 69500–69512, 2025. Disponível em: <https://doi.org/10.1109/ACCESS.2025.3561593>.

EL-SHAFEIY, E. et al. Real-time anomaly detection for water quality sensor monitoring based on multivariate deep learning technique. *Sensors*, Basileia, v. 23, n. 20, p. 8613, out. 2023. Disponível em: <https://doi.org/10.3390/s23208613>.

HUIJBREGTS, L. et al. Energy-efficient snn architecture using 3nm finfet multiport sram-based cim with online learning. In: *Proceedings of the 61st ACM/IEEE Design Automation Conference*. New York, NY, USA: Association for Computing Machinery, 2024. (DAC '24). Disponível em: <https://doi.org/10.1145/3649329.3656514>.

JANE, S. F. et al. Widespread deoxygenation of temperate lakes. *Nature*, v. 594, n. 7861, p. 66–70, 2021. Disponível em: <https://doi.org/10.1038/s41586-021-03550-y>.

JR., S. G. A. Spiking neural networks: The future of brain-inspired computing. *International Journal of Engineering Trends and Technology*, v. 73, n. 10, p. 32–48, 2025. Disponível em: <https://ijettjournal.org/Volume-73/Issue-10/IJETT-V73I10P104.pdf>.

KALAIVANAN, K.; VELLINGIRI, J. Survival study on different water quality prediction methods using machine learning. *Nature Environment and Pollution Technology*, Mumbai, v. 21, n. 3, p. 1259–1267, set. 2022. Disponível em: <https://doi.org/10.46488/NEPT.2022.v21i03.032>.

LAMRINI, M. et al. Towards reliability in smart water sensing technology: Evaluating classical machine learning models for outlier detection. *Sensors*, Basileia, v. 24, n. 13, p. 4084, jun. 2024. Disponível em: <https://doi.org/10.3390/s24134084>.

LIU, P. et al. Analysis and prediction of water quality using lstm deep neural networks in iot environment. *Sustainability*, Basileia, v. 11, n. 7, p. 2058, abr. 2019. Disponível em: <https://doi.org/10.3390/su11072058>.

LOWE, M.; QIN, R.; MAO, X. A review on machine learning, artificial intelligence, and smart technology in water treatment and monitoring. *Water*, Basileia, v. 14, n. 9, p. 1384, abr. 2022. Disponível em: <https://doi.org/10.3390/w14091384>.

MILLER, T. et al. Integrating artificial intelligence agents with the internet of things for enhanced environmental monitoring: Applications in water quality and climate data. *Electronics*, Basileia, v. 14, n. 4, p. 696, fev. 2025. Disponível em: <https://doi.org/10.3390/electronics14040696>.

NOGUEIRA, E. C. L.; RIBEIRO, J. C.; RIBEIRO, J. W. S. A educação quilombola e a luta em prol do meio ambiente na comunidade sagrado coração de jesus lago de serpa itacoatiara-amazonas. *Brazilian Journal of Development*, Curitiba, v. 8, n. 9, p. 63939–63953, set. 2022. Disponível em: <https://doi.org/10.34117/bjdv8n9-235>.

PALANI, S.; LIONG, S.-Y.; TKALICH, P. An ann application for water quality forecasting. *Marine Pollution Bulletin*, Oxford, v. 56, n. 9, p. 1586–1597, set. 2008. Disponível em: <https://doi.org/10.1016/j.marpolbul.2008.05.021>.

PAN, D. et al. Recent advances in remote sensing and artificial intelligence for river water quality forecasting: A review. *Environments*, v. 12, n. 5, p. 158, 2025. Disponível em: <https://www.mdpi.com/2076-3298/12/5/158>.

PYO, J. et al. Long short-term memory models of water quality in inland water environments. *Water Research X*, Oxford, v. 21, p. 100207, dez. 2023. Disponível em: <https://doi.org/10.1016/j.wroa.2023.100207>.

QI, C.; HUANG, S.; WANG, X. Monitoring water quality parameters of taihu lake based on remote sensing images and lstm-rnn. *IEEE Access*, Nova York, v. 8, p. 188062–188074, out. 2020. Disponível em: <https://doi.org/10.1109/ACCESS.2020.3030878>.

RAHU, M. A. et al. Toward design of internet of things and machine learning-enabled frameworks for analysis and prediction of water quality. *IEEE Access*, Nova York, v. 11, p. 1–15, set. 2023. Disponível em: <https://doi.org/10.1109/ACCESS.2023.3315649>.

RESOLUçãO CONAMA nº 357/2005: Classificação dos corpos de água e diretrizes ambientais para o enquadramento, bem como condições e padrões de lançamento de efluentes. [S.l.], 2005. Disponível em: <https://www.mpce.mp.br/wp-content/uploads/2015/12/Resolu%C3%A7%C3%A3o-CONAMA-n%C2%BA-357-05-Classifica%C3%A7%C3%A3o-dos-corpos-de-%C3%A1gua-e-as-condi%C3%A7%C3%B5es-e-padr%C3%B5es-de-lan%C3%A7amento-de-efluentes.pdf>.

ROSERO-MONTALVO, P. D. et al. Intelligent wsn system for water quality analysis using machine learning algorithms: A case study (tahuando river from ecuador). *Remote Sensing*, v. 12, n. 12, p. 1988, 2020. Disponível em: <https://www.mdpi.com/2072-4292/12/12/1988>.

WU, J.; WANG, Z. A hybrid model for water quality prediction based on an artificial neural network, wavelet transform, and long short-term memory. *Water*, Basileia, v. 14, n. 4, p. 610, fev. 2022. Disponível em: <https://doi.org/10.3390/w14040610>.

YAN, X. et al. A comprehensive review of machine learning for water quality prediction over the past five years. *Journal of Marine Science and Engineering*, Basileia, v. 12, n. 1, p. 159, jan. 2024. Disponível em: <https://doi.org/10.3390/jmse12010159>.

ZULKIFLI, C. Z. et al. Iot-based water monitoring systems: A systematic review. *Water*, Basileia, v. 14, n. 21, p. 3621, nov. 2022. Disponível em: <https://doi.org/10.3390/w14213621>.